# Cough Sound-Based Respiratory Disease Detection

**Deep Learning Project**
**Information Technology**
*Final Report*

Submitted By


| | |
|---|---|
| **Mohammad Hamza Alam** | **230911051** |
| **Devesh Gautam** | **230911360** |
| **Kummarasani Varun** | **230911350** |
| **Kanwarpal Singh Sachdeva** | **230953296** |




MANIPAL
ACADEMY of HIGHER EDUCATION
*(Institution of Eminence Deemed to be University)*

**DEPARTMENT OF INFORMATION TECHNOLOGY**
**MANIPAL INSTITUTE OF TECHNOLOGY,**
**MANIPAL ACADEMY OF HIGHER EDUCATION**

# Cough Sound-Based Respiratory Disease Detection

Mohammad Hamza Alam, Devesh Gautam, Kummarasani Varun, Kanwarpal Singh Sachdeva
Department of Information Technology, Manipal Institute of Technology, India

***Abstract*— *Cough recordings are a cheap, non-invasive signal for remote respiratory screening, but many published COVID-19 cough classifiers were evaluated on small or confounded data. We compare four models from different architecture families on the same 328 recordings from the public Coswara database, sampled so that healthy and COVID-positive participants come from the same collection batches: an MFCC + SVM baseline (M1), a 2D-CNN on log-Mel spectrograms (M2), a CNN–LSTM (M3) and a transfer-learning model using frozen YAMNet embeddings pretrained on AudioSet with a BiLSTM head (M4). Hyperparameters are chosen by nested cross-validation, so test data is never used for any choice. All four models perform above chance but modestly: 5-fold AUC is 0.62 ± 0.08 (M1), 0.61 ± 0.06 (M2), 0.56 ± 0.07 (M3) and 0.56 ± 0.09 (M4), and no deep model significantly beats the MFCC baseline. Replacing average with max pooling turned M2 from chance level (AUC 0.50) into a competitive model. Error analysis shows that the models make largely different errors (Cohen's κ 0.04–0.23), so averaging them gives the best out-of-fold AUC (0.65), that most persistent errors are missed positives, and that age and gender differ between classes, a residual confound. A noise-robustness test shows the CNN degrades least. The results support the recent view that cough audio alone carries a weak COVID-19 signal on realistic data, and that careful evaluation matters more than model complexity.*

***Keywords*— *Cough Sound Analysis, COVID-19 Screening, Deep Learning, CNN–LSTM, Transfer Learning, YAMNet, Log-Mel Spectrograms, Nested Cross-Validation***

## I. Introduction

Respiratory diseases such as asthma, chronic obstructive pulmonary disease, pneumonia and viral infections remain a major global health burden. Many cases are diagnosed late because chest X-rays, pulmonary function tests and laboratory tests are expensive and depend on clinical infrastructure. Cough is a common symptom of most of these conditions, and its acoustic properties change with the underlying disease. Because a cough can be recorded on any smartphone, it is an attractive signal for low-cost remote screening.

During the COVID-19 pandemic many studies reported very high accuracy for cough-based COVID-19 detection [2], [3], [7]. Later work showed that much of this performance does not survive on larger, better-controlled data [9], [10], [18]: models can learn who was recorded when, on which device, or which participants had symptoms, instead of the acoustic signature of the disease. This project therefore treats evaluation as seriously as modelling.

We compare four models from different architecture families on the same data, preprocessing, splits and metrics: a classical baseline (MFCC + SVM), a convolutional network (2D-CNN), a convolutional–recurrent hybrid (CNN–LSTM), and a transfer-learning model that uses a network pretrained on two million general audio clips (YAMNet + BiLSTM). Each comparison isolates one design choice: handcrafted vs. learned features (M1 vs. M2), temporal modelling (M2 vs. M3), and pretraining (M3 vs. M4). The contributions are: (1) a reproducible pipeline on the public Coswara database with a sampling scheme that removes the recording-period confound; (2) a like-for-like comparison of the four models with hyperparameters chosen by nested cross-validation; and (3) an error analysis and noise-robustness test that explain where and why the models fail.

## II. Literature Review

**Handcrafted vs. learned features.** Early cough classifiers used Mel-frequency cepstral coefficients (MFCCs) with classical classifiers such as support vector machines [11]. AI4COVID-19 [2] combined a CNN on Mel spectrograms with an MFCC + SVM branch and found the CNN branch more accurate on a small dataset, and Pahar et al. [7] compared seven classifiers on the same features, with ResNet50 best on Coswara. Our M1 vs. M2 comparison repeats this question under a stricter protocol.

**Temporal modelling.** A cough is a short sequence of phases (explosive onset, intermediate and voiced phases), so recurrent networks [12] are a natural fit. Balamurali et al. [1] used a BiLSTM on MFCC frames to separate healthy from pathological children's coughs, and LSTMs were the best model on the lab-confirmed set in [7]. This motivates M3, which places an LSTM on top of convolutional features.

**Pretraining and transfer learning.** Large audio datasets such as AudioSet [13] made general-purpose audio networks possible, including VGGish [14], PANNs [15] and YAMNet, a MobileNet [16] trained on AudioSet. Laguarta et al. [3] and Pahar et al. [8] showed that pretrained feature extractors help when labelled cough data is scarce, and Brown et al. [4] used VGGish embeddings on crowdsourced data. M4 tests this with frozen YAMNet embeddings.

**Data and evaluation.** Public datasets include Coswara [5] and COUGHVID [6]. Reported accuracies range from 73% to 98% [10], but larger and more carefully controlled studies report much lower figures: an ensemble on 1,502 participants reached AUC 0.77 [9], and a large UK study found that audio classifiers gave no clear benefit over a symptom questionnaire once confounders were controlled [18]. This is why we balance classes within collection batches, select hyperparameters without touching test data, and report cross-validated AUC with confidence intervals. Augmentation follows SpecAugment [17].

Table I summarises the core studies and how each relates to our models.

| # | Paper | Method | Dataset | Key result | Relevance to this project |
|---|---|---|---|---|---|
| [1] | Balamurali et al., *Sensors*, 2021 | MFCC (+Δ, ΔΔ) + 2-layer BiLSTM | Hospital smartphone coughs of children: healthy, asthma, URTI, LRTI | Healthy vs. pathological: AUC 0.91 (subject level) | Recurrent model on noisy cough audio (M3) |
| [2] | Imran et al., *Inform. Med. Unlocked*, 2020 | CNN on Mel spectrograms + MFCC/SVM branch with mediator | 543 coughs, 4 classes (70 COVID) | Binary CNN 92.85%; MFCC+SVM 88.76% | CNN vs. MFCC-SVM (M1 vs. M2) |
| [3] | Laguarta et al., *IEEE OJEMB*, 2020 | Three pretrained ResNet50 biomarker branches | 5,320 subjects, MIT Open Voice | AUC 0.97 | Pretrained extractors (M4) |
| [4] | Brown et al., *ACM KDD*, 2020 | Handcrafted + VGGish features, LR/SVM | COVID-19 Sounds (crowdsourced) | AUC > 0.80 | Pretrained embeddings on crowdsourced data (M4) |
| [5] | Sharma et al., *Interspeech*, 2020 | Coswara dataset + baseline | Crowdsourced cough, breath, voice | Open dataset | Dataset used in this project |
| [6] | Orlandic et al., *Sci. Data*, 2021 | COUGHVID dataset, XGBoost cough detector | 25,000+ recordings | Cough detector AUC 0.964 | Candidate external test set |
| [7] | Pahar et al., *Comput. Biol. Med.*, 2021 | 7 classifiers incl. SVM, CNN, LSTM, ResNet50 | Coswara + lab-confirmed set | ResNet50 AUC 0.98; LSTM 0.94 | Closest design: many models, one pipeline |
| [8] | Pahar et al., *Comput. Biol. Med.*, 2022 | Pretraining + fine-tuning (CNN, LSTM, ResNet50) | Several cough/breath/speech sets | AUC 0.98 (cough) | Transfer learning (M4) |
| [9] | Mohammed et al., *Sci. Rep.*, 2021 | 6 feature types; shallow + deep models; ensemble | 1,502 participants | Ensemble AUC 0.77 | Lower, realistic performance on larger data |
| [10] | Ghrabli et al., *Diagnostics*, 2022 | Scoping review of 35 DL studies | 16 to 30,000+ samples | Accuracy 73.1–98.5%; inconsistent metrics | Our evaluation rules |
| [15] | Kong et al., *IEEE/ACM TASLP*, 2020 | PANNs: CNNs pretrained on AudioSet | AudioSet (2M clips) | mAP 0.439 on AudioSet | Large-scale audio pretraining (M4) |
| [18] | Coppock et al., *Nat. Mach. Intell.*, 2024 | Audio classifiers vs. symptom checker, confounder-matched | UK COVID-19 Vocal Audio Dataset | No clear gain over symptoms after matching | Motivates confound control |

*Table I. Summary of the core literature*

## III. Objectives

1. Build a reproducible, confound-controlled pipeline for cough-based COVID-19 detection on the public Coswara database.
2. Implement four models from different architecture families (classical, CNN, CNN–RNN, transfer learning) and compare them on the same splits and metrics, with hyperparameters selected without access to test data.

3. Explain the differences through error analysis (by severity, subgroup and recording properties) and a noise-robustness test.

## IV. Dataset and Preprocessing

### A. Dataset

We use the *heavy cough* recording of each participant in Coswara [5] and the self-reported COVID-19 status from its metadata: *healthy* → class 0; *positive* (asymptomatic, mild, moderate) → class 1; other statuses are excluded. In Coswara most healthy participants were recorded in 2020 and most positives in 2021–22, so a naive sample would let a model detect the recording period. We therefore use only the 15 collection batches from 2021–22 and, within each batch, randomly sample equally many healthy and positive participants (seed 42). After removing unreadable, silent or very short files, the dataset has **328 recordings from 328 participants** (29 asymptomatic, 99 mild, 36 moderate, 164 healthy). One recording per participant means no speaker appears in two splits.

| Split | Healthy (0) | COVID (1) | Total |
|---|---|---|---|
| Train | 114 | 115 | 229 |
| Validation | 25 | 24 | 49 |
| Test | 25 | 25 | 50 |
| Total | 164 | 164 | 328 |

*Table II. Fixed stratified 70/15/15 split*

### B. Preprocessing

All models share one pipeline: mono conversion, resampling to 16 kHz, stationary spectral gating to suppress background noise, energy-based voice-activity detection (frames with RMS above 1.5 × the median RMS, with 0.1 s padding) to keep the cough region, and padding or cropping to 4 s. M1–M3 use z-score amplitude normalisation; M4 uses peak normalisation because YAMNet expects waveforms in [−1, 1]. Features: **M1** 13 MFCCs with Δ and ΔΔ, summarised by mean and standard deviation (78 values); **M2/M3** a 128-band log-Mel spectrogram (n_fft 512, hop 160 = 10 ms, 128 × 401), standardised with training statistics; **M4** eight 1,024-dimensional YAMNet embeddings (0.96 s windows, 0.48 s hop). Training-time augmentation for M2/M3 is a random time shift (±0.4 s), Mel-bin shift (±2 bins) and Gaussian noise; SpecAugment masking [17] is a tuning option. Fig. 1 shows the pipeline.

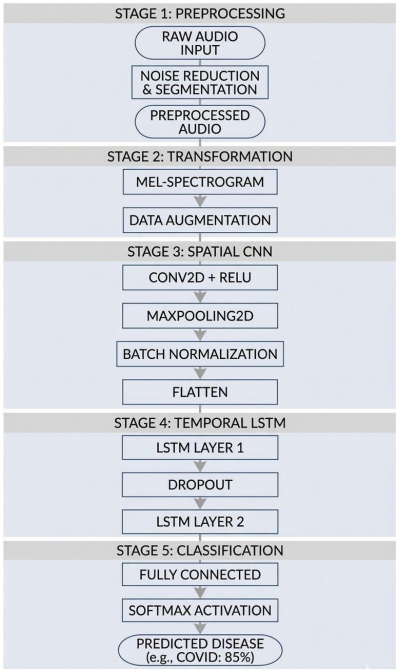



*Fig. 1. Processing pipeline (shown for the CNN–LSTM)*

## V. Methodology

The four models were implemented by different team members; each subsection names its owner. All deep models (M2–M4) share one training loop so that differences come from the architecture, not the training.

*A. M1: MFCC + Support Vector Machine (owner: Mohammad Hamza Alam)*

M1 is the classical baseline. MFCCs describe the spectral envelope of each 25 ms frame on a perceptual (Mel) scale; their first and second differences describe how the envelope changes. Taking the mean and standard deviation over time gives a fixed 78-dimensional vector per recording, but discards the order of events. After standardisation, an SVM with a radial basis function kernel [11] finds the maximum-margin boundary in a non-linear feature space. C controls how strongly misclassified training points are penalised and γ the width of the kernel; both were tuned (Section VI). Because it needs no early stopping, the final M1 is refit on training + validation data. Selected setting: C=10.0, gamma=0.001.

*B. M2: 2D-CNN on Log-Mel Spectrograms (owner: Kummarasani Varun)*

M2 treats the spectrogram as an image. Two convolutional blocks (3×3 convolution, ReLU, 2×2 max pooling, batch normalisation; 32 and 64 channels) turn the 1 × 128 × 401 input into a 64 × 32 × 100 feature map. A global pooling layer then reduces each channel to one number, followed by dropout (0.5) and a linear layer, giving 19,138 parameters. In Phase 2 the global pooling was an average, and the network did not learn: averaging over 4 s mixes a short cough burst with silence and padding. Global max pooling, which keeps each channel's strongest response, was added as a tuning option. M2 deliberately has no temporal model; it measures what convolution alone can do. Selected setting: max pooling, lr=0.001.

*C. M3: CNN–LSTM (owner: Kanwarpal Singh Sachdeva)*

M3 uses the same convolutional front-end as M2 but keeps the time axis. The 64 × 32 × 100 feature map is rearranged into a sequence of 100 time steps, each a 2,048-dimensional vector (64 channels × 32 Mel bins). A stacked LSTM [12] (128 units returning the full sequence, dropout 0.5, then 64 units) reads this sequence in order; its gates decide what to remember about earlier frames, so the model can represent the onset, decay and rhythm of a cough. The last hidden state feeds a linear layer. M3 has 1,183,938 parameters, most of them in the first LSTM. Because M2 and M3 differ only after the convolutional blocks, their difference measures the value of temporal modelling. Selected setting: lr=0.0003, SpecAugment=on.

*D. M4: Pretrained YAMNet + BiLSTM (owner: [owner])*

M4 tests transfer learning. YAMNet is a MobileNet-v1 CNN [16] trained by Google on AudioSet [13] (about 2 million YouTube clips, 521 sound classes including *Cough*, *Sneeze* and *Throat clearing*). We use the official TensorFlow Hub model in TFLite form, frozen, as a feature extractor: it computes its own log-Mel front-end and outputs a 1,024-dimensional embedding for each 0.96 s window, giving 8 embeddings per 4 s recording. As a sanity check, YAMNet's most frequent top AudioSet class over our recordings is *Cough* (114 of 328; most others are *Silence*, caused by the padding, or *Speech*).Only a small head is trained on our data: a bidirectional LSTM (64 units per direction) over the 8 embeddings, averaged over time, then a linear layer (558,338 trainable parameters). Two order-free heads (mean or max over time, then a two-layer MLP) were tuning alternatives. The idea is that features learned from millions of sounds need far fewer labelled examples than features learned from scratch. Selected setting: bilstm head, lr=0.0003.

## VI. Experimental Setup

**Training.** M2–M4 use Adam (weight decay $10^{-4}$), batch size 16, cross-entropy loss, at most 40 epochs and early stopping on validation loss (patience 8), restoring the best checkpoint. Seeds are fixed (42). All experiments ran on a 2-core CPU with PyTorch, scikit-learn, librosa and the LiteRT (TFLite) runtime.

**Hyperparameter selection (nested).** Each model has a small grid (Table IV). For every setting we run stratified 5-fold cross-validation over all 328 recordings; inside each fold, 15% of the training portion is held out as an inner validation set for early stopping and selection. The setting with the highest mean *inner-validation* AUC is selected, so the outer test folds are never used for any choice. The selected setting's outer-fold results are the reported cross-validation results, and the same setting is then trained on the fixed split (Table II).

**Metrics.** Accuracy, AUC (threshold-free), sensitivity (recall on COVID), specificity (recall on healthy) and macro F1, at a 0.5 threshold. Because one error on the 50-recording test set moves accuracy by 2 percentage points, cross-validated and out-of-fold (OOF) results are the primary basis for comparison. OOF predictions give every recording exactly one prediction from a model that never saw it; we report 95% bootstrap confidence intervals (2,000 resamples) and a paired bootstrap for AUC differences.

| Model | Settings searched |
|---|---|
| M1 | C {0.1, 1, 10} × γ {scale, 0.001, 0.01} (9) |
| M2 | global average pooling, lr $10^{-3}$; global max pooling, lr {$10^{-3}$, $3{\cdot}10^{-4}$} (3) |
| M3 | lr $10^{-3}$; lr $3{\cdot}10^{-4}$; lr $3{\cdot}10^{-4}$ + SpecAugment (3) |
| M4 | head {BiLSTM, mean-pool MLP, max-pool MLP} × lr {$10^{-3}$, $3{\cdot}10^{-4}$} (6) |

*Table III. Hyperparameter grids*

## VII. Results and Discussion

### *A. Comparison of the Four Models*

| Model | Selected setting | Test acc. | Test AUC | Sens. | Spec. | Macro F1 | 5-fold AUC | OOF AUC [95% CI] |
|---|---|---|---|---|---|---|---|---|
| M1 MFCC + SVM | C=10.0, gamma=0.001 | 64% | 0.70 | 0.52 | 0.76 | 0.63 | 0.62 ± 0.08 | 0.62 [0.55, 0.68] |
| M2 2D-CNN | max pooling, lr=0.001 | 62% | 0.67 | 0.52 | 0.72 | 0.62 | 0.61 ± 0.06 | 0.59 [0.53, 0.65] |
| M3 CNN–LSTM | lr=0.0003, SpecAugment=on | 58% | 0.64 | 0.48 | 0.68 | 0.58 | 0.56 ± 0.07 | 0.57 [0.51, 0.63] |
| M4 YAMNet + BiLSTM | bilstm head, lr=0.0003 | 52% | 0.48 | 0.48 | 0.56 | 0.52 | 0.56 ± 0.09 | 0.56 [0.49, 0.62] |

*Table IV. Main comparison. Test = fixed split (n = 50); 5-fold = mean ± std over outer folds; OOF = all 328 out-of-fold predictions*

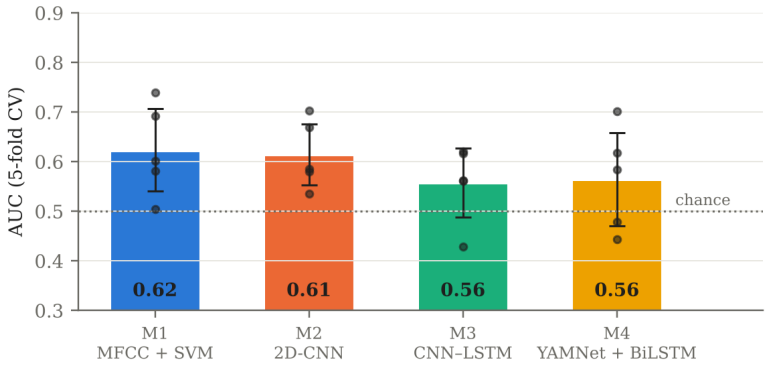



*Fig. 2. 5-fold AUC per model (bars: mean, whiskers: std, dots: folds)*

Table IV and Fig. 2 give the main comparison. On the 5-fold cross-validation, the primary measure, the MFCC + SVM baseline (M1) has the highest AUC (0.62 ± 0.08), closely followed by the 2D-CNN (M2, 0.61 ± 0.06); the CNN–LSTM (M3) and the transfer-learning model (M4) reach 0.56. Out-of-fold (OOF) AUCs agree (0.62, 0.59, 0.57, 0.56), and all 95% confidence intervals lie at or above 0.5, so every model has learned some real signal. The differences, however, are not statistically reliable: the paired bootstrap difference to M1 is −0.03 [−0.10, 0.05] for M2, −0.05 [−0.13, 0.03] for M3 and −0.06 [−0.13, 0.02] for M4. The fixed 50-recording test set gives the same ranking (test AUC 0.70, 0.67, 0.64, 0.49), but with one error moving accuracy by 2 percentage points its individual numbers are less reliable than the cross-validated ones. All models are more specific than sensitive at the 0.5 threshold (e.g. M3: sensitivity 0.38, specificity 0.73 OOF), which for screening would call for a lower threshold.

### *B. Effect of Tuning*

| Model | Setting | Inner-val AUC | Outer 5-fold AUC |
|---|---|---|---|
| M1 MFCC + SVM | C=0.1, gamma=scale | 0.475 | 0.390 ± 0.119 |

| Model | Setting | Inner-val AUC | Outer 5-fold AUC |
|---|---|---|---|
| | C=0.1, gamma=0.001 | 0.476 | 0.382 ± 0.125 |
| | C=0.1, gamma=0.01 | 0.481 | 0.394 ± 0.119 |
| | C=1.0, gamma=scale | 0.552 | 0.643 ± 0.082 |
| | C=1.0, gamma=0.001 | 0.501 | 0.535 ± 0.168 |
| | C=1.0, gamma=0.01 | 0.544 | 0.641 ± 0.080 |
| | C=10.0, gamma=scale | 0.539 | 0.649 ± 0.078 |
| | C=10.0, gamma=0.001 ← selected | 0.557 | 0.624 ± 0.083 |
| | C=10.0, gamma=0.01 | 0.542 | 0.648 ± 0.077 |
| M2 2D-CNN | avg pooling, lr=0.001 | 0.589 | 0.501 ± 0.048 |
| | max pooling, lr=0.001 ← selected | 0.608 | 0.615 ± 0.062 |
| | max pooling, lr=0.0003 | 0.414 | 0.499 ± 0.063 |
| M3 CNN–LSTM | lr=0.001, SpecAugment=off | 0.606 | 0.565 ± 0.062 |
| | lr=0.0003, SpecAugment=off | 0.649 | 0.567 ± 0.038 |
| | lr=0.0003, SpecAugment=on ← selected | 0.673 | 0.558 ± 0.069 |
| M4 YAMNet + BiLSTM | bilstm head, lr=0.001 | 0.567 | 0.570 ± 0.089 |
| | bilstm head, lr=0.0003 ← selected | 0.627 | 0.565 ± 0.094 |
| | mean head, lr=0.001 | 0.510 | 0.537 ± 0.067 |
| | mean head, lr=0.0003 | 0.500 | 0.566 ± 0.079 |
| | max head, lr=0.001 | 0.425 | 0.555 ± 0.045 |
| | max head, lr=0.0003 | 0.463 | 0.532 ± 0.098 |

*Table V. All settings searched. Selection uses only the inner-validation AUC*

Table V lists every setting. Three results stand out. First, **max pooling fixes M2**: with global average pooling, as in Phase 2, M2 stays at chance (AUC 0.50 ± 0.05); with global max pooling it reaches 0.62 ± 0.06, confirming the Phase 2 diagnosis that averaging over 4 s dilutes the short cough burst with silence. Second, **tuning did not help M3**: a lower learning rate and SpecAugment raised its inner-validation AUC (0.61 → 0.67) but not its test AUC (0.565 → 0.558). Third, the inner-validation sets hold only about 40 recordings, so selection is noisy: for M1 the selected setting (C = 10, γ = 0.001) scores 0.62 on the outer folds, slightly below the Phase 2 default (0.64). We report the selected setting anyway, because choosing by test results would be optimistic. With C = 0.1 the SVM is almost constant and its Platt-scaled probabilities become unreliable (AUC below 0.5). For M4, the BiLSTM head was selected over the order-free mean and max heads. Finally, the Phase 2 single-split result for M3 (test AUC 0.78) did not hold up under cross-validation (0.56), a concrete example of how much a 50-recording test set can mislead.

### *C. Training Behaviour and Test-Set Errors*

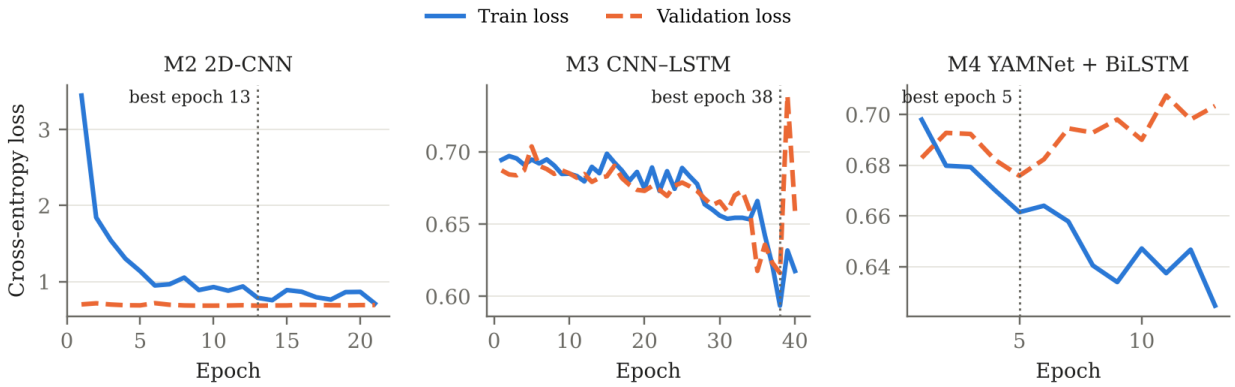



*Fig. 3. Training and validation loss on the fixed split (dotted line: restored checkpoint)*

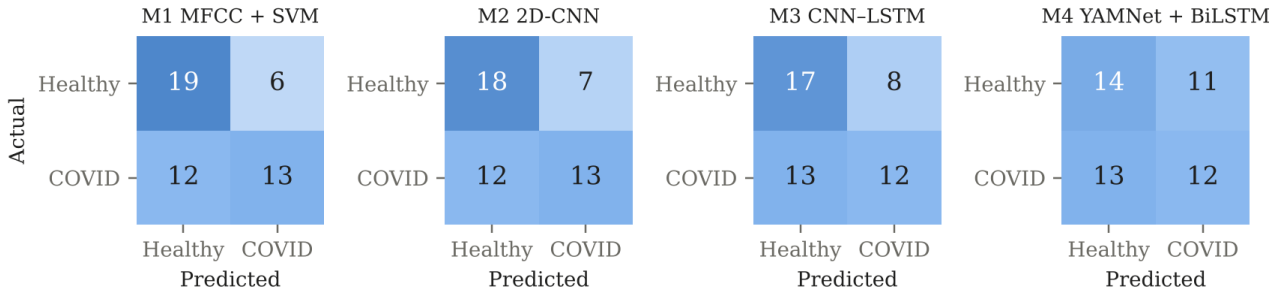



*Fig. 4. Confusion matrices on the fixed test set (n = 50)*

Fig. 3 shows the loss curves on the fixed split. M2 starts with a high training loss, because max pooling passes large activations, and converges in about 10 epochs while its validation loss barely moves. M3 learns slowly at the selected learning rate ($3{\cdot}10^{-4}$) and its best epoch (38) is close to the 40-epoch limit, followed by a sharp rise in validation loss; in cross-validation its best epochs varied from 4 to 25, another sign of high variance. M4's head fits the training data within 5 epochs and then overfits, which is expected with 558k trainable parameters on 229 examples. The confusion matrices (Fig. 4) show that all models miss about half of the COVID cases on the test set while recognising most healthy recordings.

*D. Error Analysis*

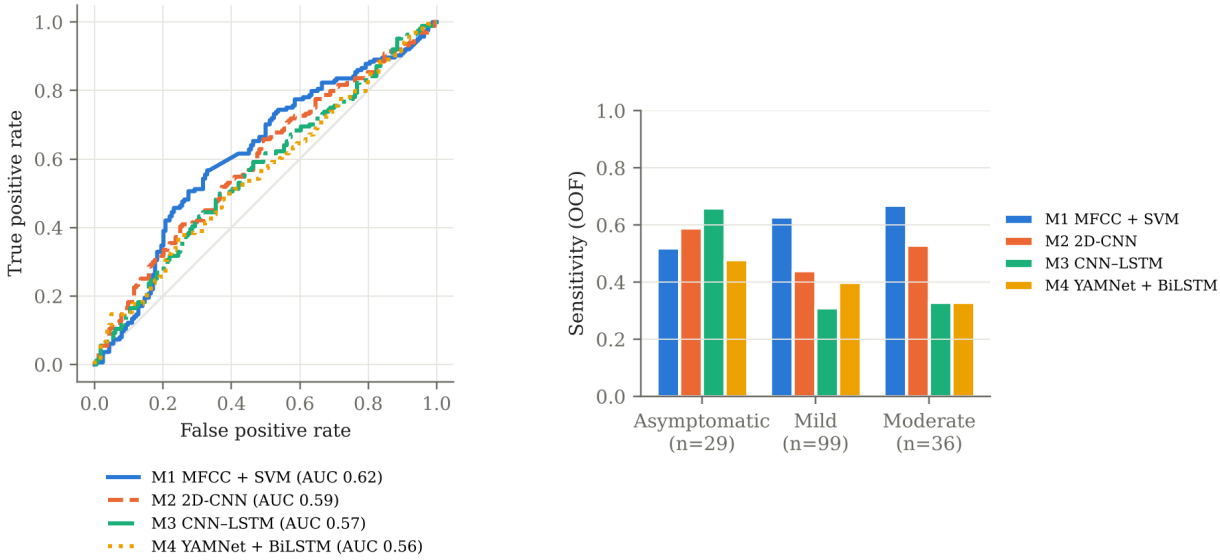



*Fig. 5. Left: ROC curves of out-of-fold predictions. Right: sensitivity by self-reported severity*

| Group | n (healthy / COVID) | M1 sens. / spec. | M2 sens. / spec. | M3 sens. / spec. | M4 sens. / spec. |
|---|---|---|---|---|---|
| Male | 116 / 99 | 0.56 / 0.60 | 0.40 / 0.65 | 0.38 / 0.72 | 0.40 / 0.67 |
| Female | 47 / 64 | 0.72 / 0.51 | 0.61 / 0.62 | 0.36 / 0.72 | 0.39 / 0.68 |
| Age < 30 | 96 / 56 | 0.59 / 0.59 | 0.52 / 0.64 | 0.38 / 0.74 | 0.30 / 0.70 |
| Age 30–45 | 47 / 54 | 0.56 / 0.57 | 0.46 / 0.66 | 0.37 / 0.68 | 0.33 / 0.68 |
| Age > 45 | 21 / 54 | 0.70 / 0.52 | 0.48 / 0.57 | 0.39 / 0.76 | 0.57 / 0.57 |

*Table VI. Out-of-fold sensitivity / specificity by subgroup*

**Severity.** Fig. 5 (right) shows OOF sensitivity by self-reported severity. Only M1 becomes more sensitive as symptoms get stronger (0.52 asymptomatic, 0.63 mild, 0.67 moderate), which is what one would expect if it detects symptom-related acoustic changes. M3 and M4 are, surprisingly, most sensitive on asymptomatic cases (0.66 and 0.48) and weakest on moderate ones (0.33), which suggests they rely on cues other than cough acoustics; with 29 asymptomatic cases these rates are uncertain. **Demographics.** Although classes are balanced

within collection batches, they are not balanced by age or gender: COVID-positive participants are older (54 of 75 participants over 45 are positive, but only 56 of 152 under 30) and more often female (64 of 111 women vs. 99 of 215 men). M1's sensitivity is highest exactly in the groups where positives are over-represented (women 0.72, over-45s 0.70) and its specificity lowest there (0.51, 0.52) (Table VI), so part of what M1 learns is likely voice characteristics linked to age and gender rather than disease. This is a residual confound that our batch matching does not remove. **Error overlap.** Agreement between models is low (Cohen's κ 0.04–0.23): only 52 recordings are classified correctly by all four models and 25 by none. Because the models fail on different recordings, averaging their probabilities gives the best OOF AUC, 0.65 [0.59, 0.71] (+0.03 over M1, not significant). **Hard cases.** Of the 90 recordings misclassified by at least three models, 69% are COVID-positive (vs. 38% of the 145 easy ones): the dominant error is a missed positive, i.e. a COVID cough that sounds normal. Hard recordings also have a longer detected sound region (median 5.6 s vs. 4.7 s), which points to recordings with extra sounds or several coughs, while their loudness is the same.

*E. Robustness to Background Noise*

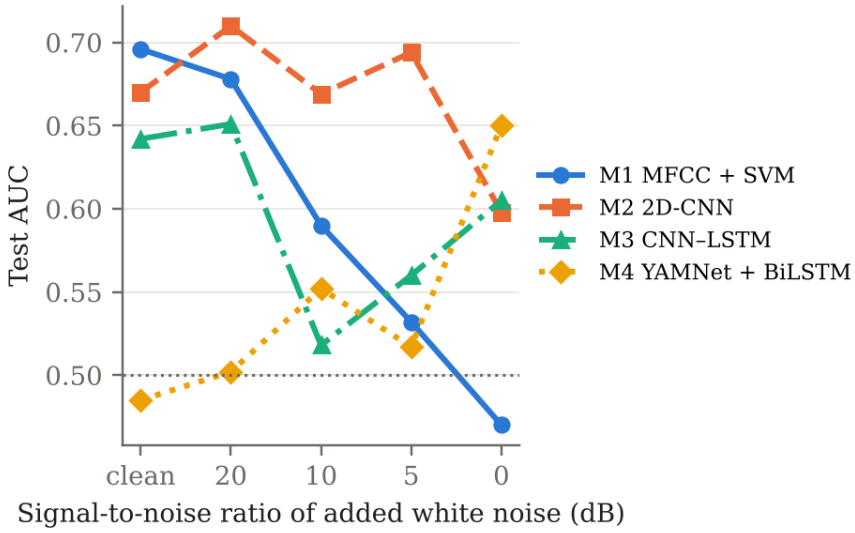



*Fig. 6. Test AUC when white noise is added to the raw recordings before preprocessing*

Fig. 6 shows test AUC as white noise is added to the raw recordings before the full preprocessing. M1 degrades steadily, from 0.70 (clean) to 0.59 at 10 dB and 0.47 at 0 dB: MFCC statistics shift directly with the noise floor, despite spectral gating. M2 is the most robust, staying between 0.67 and 0.71 down to 5 dB and falling to 0.60 at 0 dB; max pooling picks the strongest local patterns, which survive added noise. M3 fluctuates (0.52–0.65), and M4, near chance on clean audio, varies between 0.49 and 0.65 without a trend. With only 50 test recordings, differences under about 0.08 AUC should not be over-interpreted, but the steady decline of M1 and the stability of M2 are clear trends.

*F. Why the Models Differ*

**Handcrafted vs. learned features (M1 vs. M2).** With 229 training recordings, compact MFCC statistics are as good as learned convolutional features. M2 needed the right pooling to work at all, but once it did it matched M1 and was far more robust to noise, so learned features help most when recording conditions vary. **Temporal modelling (M2 vs. M3).** Adding an LSTM did not help; it added 1.16 million parameters to the same front-end, and with so few examples the extra capacity mostly increases variance (best epochs from 4 to 38). The temporal structure that motivated M3 is real, but learning it needs much more data. **Pretraining (M3 vs. M4).** Frozen YAMNet embeddings did not help either. YAMNet was trained to tell a cough from a sneeze or speech, and *Cough* is indeed its most frequent label for our recordings, so its embeddings mostly encode *that* the sound is a cough. The difference between a COVID and a healthy cough is a much subtler within-class variation that generic features do not emphasise, and averaging 8 windows that are largely silent padding dilutes it further. **Overall.** All four models are well below the 90%+ figures of early studies [2], [3] and close to what larger, controlled studies report [9], [18]. Our data shows why: once the recording-period confound is removed the remaining signal is weak, residual confounds such as age and gender still contribute to it, and on a few

hundred recordings the variance between folds (±0.06–0.09 AUC) is larger than the differences between models. In this regime, model complexity buys little, and the evaluation protocol decides whether a result can be trusted.

## VIII. Conclusion

We built a reproducible, confound-controlled pipeline for cough-based COVID-19 detection on the public Coswara database and compared four architecture families under one nested evaluation protocol. All models learned a real but weak signal (5-fold AUC 0.56–0.62); the simple MFCC + SVM baseline was best, a max-pooling 2D-CNN was statistically tied with it and the most robust to noise, and neither temporal modelling (CNN–LSTM) nor frozen pretrained embeddings (YAMNet) improved results on 229 training recordings. The models make different errors, so their average performed best (OOF AUC 0.65). The error analysis identified missed positives as the main failure and age and gender as a residual confound. Our main lesson is methodological: with small crowdsourced audio datasets, a single test split can overstate performance substantially (AUC 0.78 vs. 0.56 for the same model), and confound control and nested selection matter more than architecture. Cough audio alone is not yet a reliable COVID-19 screening tool on this data.

## IX. Future Work

- **More and better-labelled data.** Add the remaining Coswara recordings (shallow cough, breathing, speech) of the same participants, and lab-confirmed datasets, to reduce the variance that dominates our results.
- **External validation.** Train on Coswara and test on COUGHVID [6] to measure how well the models transfer across collection apps and countries.
- **Fine-tuning the pretrained network.** Unfreeze YAMNet's last blocks or use larger pretrained audio models such as PANNs [15], on a GPU.
- **Combining audio with symptoms.** Following [18], test whether audio adds information beyond a short symptom questionnaire, which is the question that matters for real screening.
- **Calibration and threshold choice.** Choose the decision threshold for a target sensitivity (for example 90%) on validation data, as a screening tool would require.


## Acknowledgement

We thank our course instructor for guidance throughout the project, and the Coswara team at the Indian Institute of Science, Bangalore, for making their dataset publicly available. YAMNet was released by Google under the Apache 2.0 licence.


## Use of AI Tools

As required by the course guidelines, we disclose the use of an AI assistant (Claude, Anthropic). It was used to draft and refactor parts of the code (data download, preprocessing, the training loop, the YAMNet wrapper, the evaluation, error-analysis and figure scripts), to draft text in this report, and to format the report and slides. All experiments were run, checked and interpreted by the team; each member is responsible for, and can explain, the model and code they own. [Team: edit this paragraph so it states exactly where AI was and was not used, before submitting.]

## References


[1] B. T. Balamurali, H. I. Hee, S. Kapoor, O. H. Teoh, S. S. Teng, K. P. Lee, D. Herremans, and J. M. Chen, "Deep neural network-based respiratory pathology classification using cough sounds," *Sensors*, vol. 21, no. 16, p. 5555, 2021.

[2] A. Imran, I. Posokhova, H. N. Qureshi, U. Masood, M. S. Riaz, K. Ali, C. N. John, M. I. Hussain, and M. Nabeel, "AI4COVID-19: AI enabled preliminary diagnosis for COVID-19 from cough samples via an app," *Informatics in Medicine Unlocked*, vol. 20, p. 100378, 2020.

[3] J. Laguarta, F. Hueto, and B. Subirana, "COVID-19 artificial intelligence diagnosis using only cough recordings," *IEEE Open Journal of Engineering in Medicine and Biology*, vol. 1, pp. 275–281, 2020.

[4] C. Brown, J. Chauhan, A. Grammenos, J. Han, A. Hasthanasombat, D. Spathis, T. Xia, P. Cicuta, and C. Mascolo, "Exploring automatic diagnosis of COVID-19 from crowdsourced respiratory sound data," in *Proc. 26th ACM SIGKDD*, 2020, pp. 3474–3484.

[5] N. Sharma *et al.*, "Coswara – A database of breathing, cough, and voice sounds for COVID-19 diagnosis," in *Proc. Interspeech*, 2020, pp. 4811–4815.

[6] L. Orlandic, T. Teijeiro, and D. Atienza, "The COUGHVID crowdsourcing dataset, a corpus for the study of large-scale cough analysis algorithms," *Scientific Data*, vol. 8, p. 156, 2021.

[7] M. Pahar, M. Klopper, R. Warren, and T. Niesler, "COVID-19 cough classification using machine learning and global smartphone recordings," *Computers in Biology and Medicine*, vol. 135, p. 104572, 2021.

[8] M. Pahar, M. Klopper, R. Warren, and T. Niesler, "COVID-19 detection in cough, breath and speech using deep transfer learning and bottleneck features," *Computers in Biology and Medicine*, vol. 141, p. 105153, 2022.

[9] E. A. Mohammed, M. Keyhani, A. Sanati-Nezhad, S. H. Hejazi, and B. H. Far, "An ensemble learning approach to digital corona virus preliminary screening from cough sounds," *Scientific Reports*, vol. 11, 2021.

[10] S. Ghrabli, M. Elgendi, and C. Menon, "Challenges and opportunities of deep learning for cough-based COVID-19 diagnosis: A scoping review," *Diagnostics*, vol. 12, no. 9, p. 2142, 2022.

[11] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio Set: An ontology and human-labeled dataset for audio events," in *Proc. IEEE ICASSP*, 2017, pp. 776–780.

[14] S. Hershey *et al.*, "CNN architectures for large-scale audio classification," in *Proc. IEEE ICASSP*, 2017, pp. 131–135.

[15] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "PANNs: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.

[16] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[17] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019, pp. 2613–2617.

[18] H. Coppock *et al.*, "Audio-based AI classifiers show no evidence of improved COVID-19 screening over simple symptoms checkers," *Nature Machine Intelligence*, vol. 6, pp. 229–242, 2024.

## APPENDIX A: CODE AND REPRODUCIBILITY

All results in this report were produced by the code in the project repository (github.com/21hamza/cough-respiratory-disease-detection), written in Python with PyTorch, scikit-learn, librosa and the LiteRT (TFLite) runtime. A single entry point, `main.py`, runs the complete pipeline from raw audio to the figures, or any single stage. Each stage writes its console output to `logs/`, and the repository contains the logs of the runs reported here. Seeds are fixed (42), so the stages reproduce the reported numbers exactly.

| Stage | Command | Code | Output |
|---|---|---|---|
| Download data | python main.py --stage download | src/data/download_coswara.py | data/raw/ |
| Build dataset | python main.py --stage dataset | src/data/build_dataset.py, preprocessing.py | splits, MFCC and log-Mel cache |
| YAMNet features | python main.py --stage yamnet | src/data/extract_yamnet.py, models/m4_yamnet.py | YAMNet embeddings |
| Tuning (nested CV) | python main.py --stage tune | src/run_experiment.py, experiments.py, training.py | results/cv/ (Tables IV, V) |
| Final training | python main.py --stage train | src/models/m1_svm.py … m4_yamnet.py | results/main/, checkpoints (Fig. 3, 4) |
| Error analysis | python main.py --stage analyse | src/error_analysis.py | Table VI, Fig. 5 |
| Robustness | python main.py --stage robust | src/robustness.py | Fig. 6 |
| Figures | python main.py --stage figures | src/make_figures.py | figures/, tables |
| Inference | python predict.py file.wav | predict.py | P(COVID) from each model |

*Table A1. Pipeline stages, commands and source files (run time about 2.5 h on a 2-core CPU)*

## Individual Contribution Statement

Each member lists the work they did. All members have read the report and sign below.

| Member Name | Reg. No. | Contribution | Signature |
|---|---|---|---|
| Mohammad Hamza Alam | 230911051 | | |
| Devesh Gautam | 230911360 | | |
| Kummarasani Varun | 230911350 | | |
| Kanwarpal Singh Sachdeva | 230953296 | | |

Code repository: github.com/21hamza/cough-respiratory-disease-detection